



**BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE
APPLICATIONS AND GEO-INFORMATICS**

WEEKLY PROGRESS REPORT (12/03/2026 – 18/03/2026)

WEEK 11

PROJECT NAME	Monitoring and change detection system using OSINT
---------------------	---

PROJECT DESCRIPTION :

**CREATING OSINT TOOL FOR MONITORING AND
CHANGE DETECTION SYSTEM BY MONITORING
PUBLICLY AVAILABLE INFORMATION.**

GROUP MEMBER :

Harshil Vaniya, Shalomo Agarwarkar, Vrushank Thakkar

GROUP ID :

2026G195

GROUP GUIDE :

CHASIYA KRUTIKA

GROUP COORDINATOR :

SIDHDHARTH PATEL

COLLEGE GUIDE NAME :

BHUMIKA DOSHI

COLLEGE GUIDE No. :

9016888925

COLLEGE GUIDE EMAIL. :

BHUMIKASDOSHI@GMAIL.COM

COLLEGE NAME :

**DEPT. OF BIOCHEMISTRY AND FORENSICS SCIENCE, GUJARAT
UNIVERSITY**

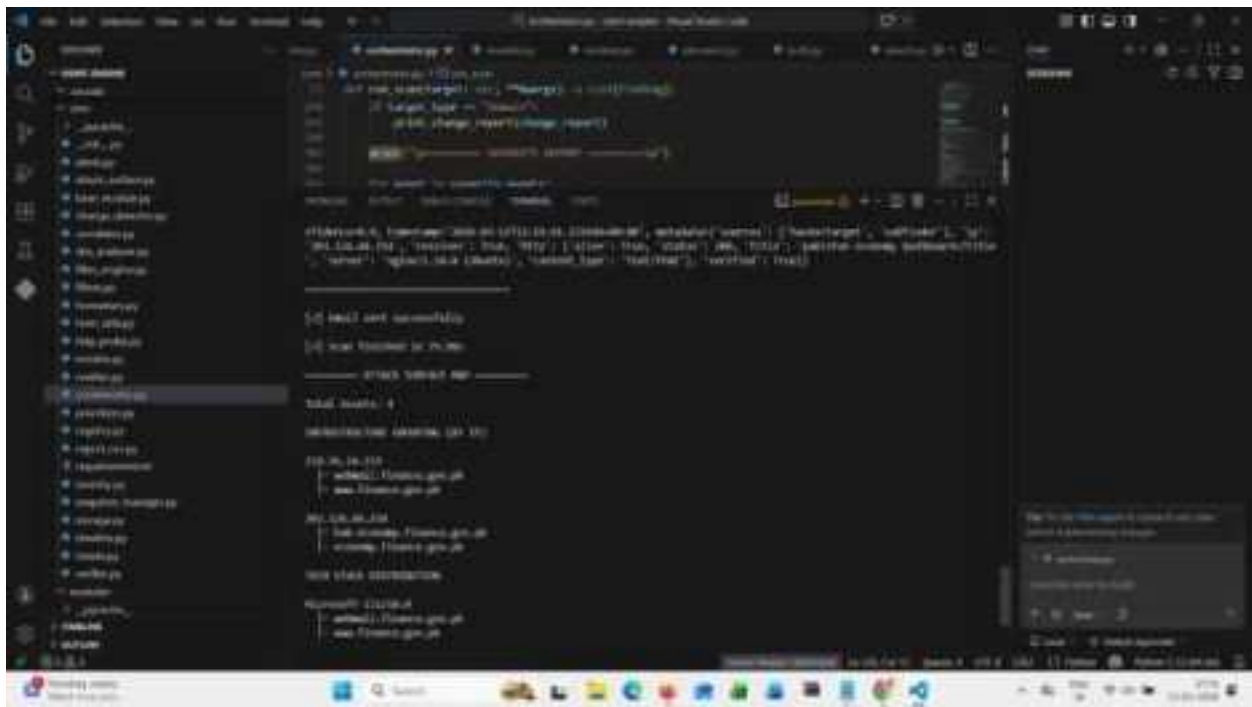
12/03/2026	- Reviewed project structure and planned upcoming tasks for further development and testing (Harshil Vaniya) - find an alternative to the email osint tool ghunt and created code for it and testing (Vrushank Thakkar, Shalomo Agarwarkar)
13/03/2026	- trying to integrating mr holmes into current osint tool and error solving (Vrushank Thakkar) - searched for public-facing APIs available for email OSINT and how to integrate them (Harshil Vaniya) - Studied the dehash security leak checker platform. (Shalomo Agarwarkar)
14/03/2026	2nd Saturday (Holiday)
15/03/2026	Holiday (Sunday)
16/03/2026	- created a module of usernames using Sherlock, WhatsMyName, and a personal one, and checked the usernames on many sites. (Vrushank Thakkar, Harshil Vaniya) - Studied network request of " Have I been Pawned. (Shalomo Agarwarkar)
17/03/2026	- integrated modules (Harshil Vaniya) - username module integration (Vrushank Thakkar) - Tested the HIBP tool and constructed a social media account verifier based on email. (Shalomo Agarwarkar)
18/03/2026	- fixed results formatting errors for the email and username module. (Vaniya Harshil, Vrushank Thakkar) - Tested and integrated email, reputation chapter. (Shalomo Agarwarkar, Vrushank Thakkar)

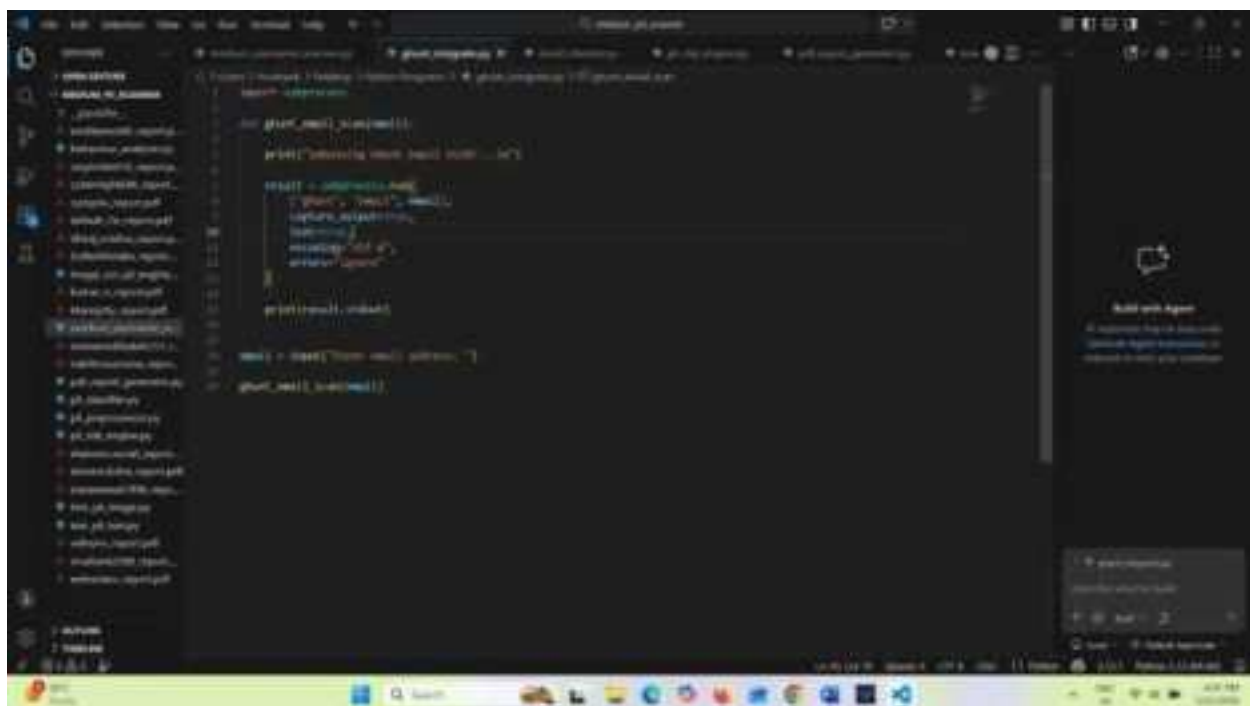
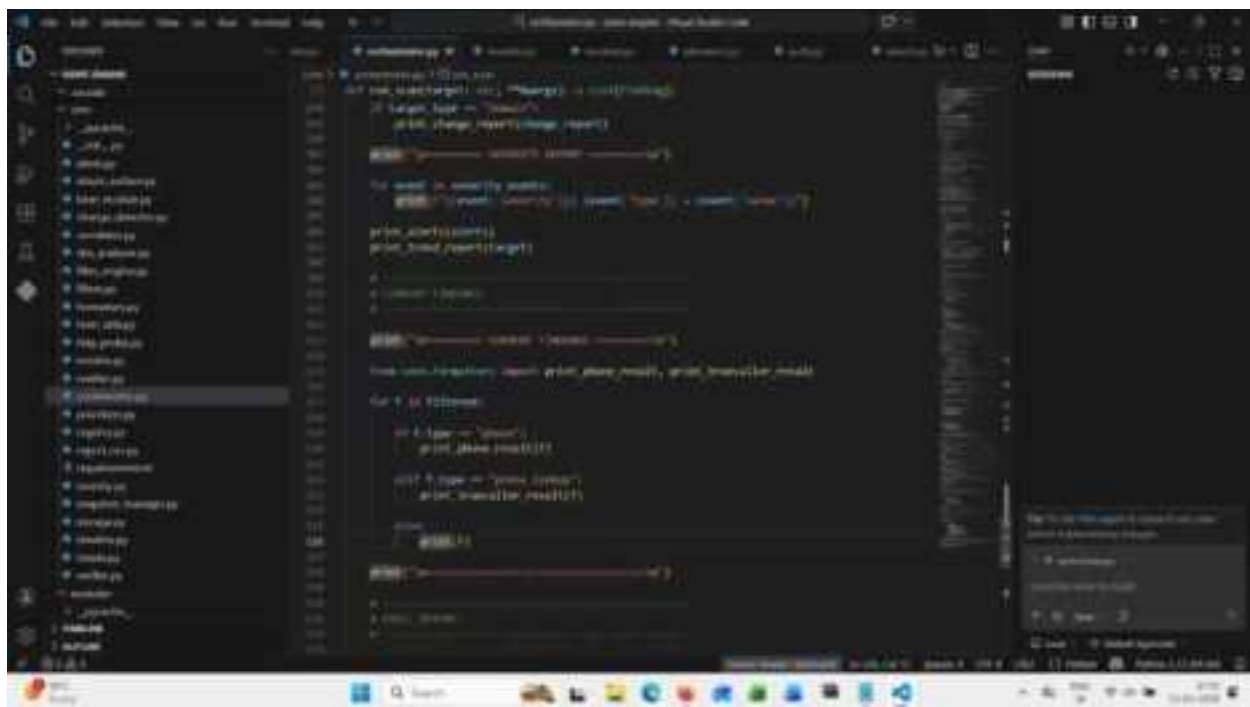
Next Week Plan	Improving email and username modules and optimizing results formatting
-----------------------	---

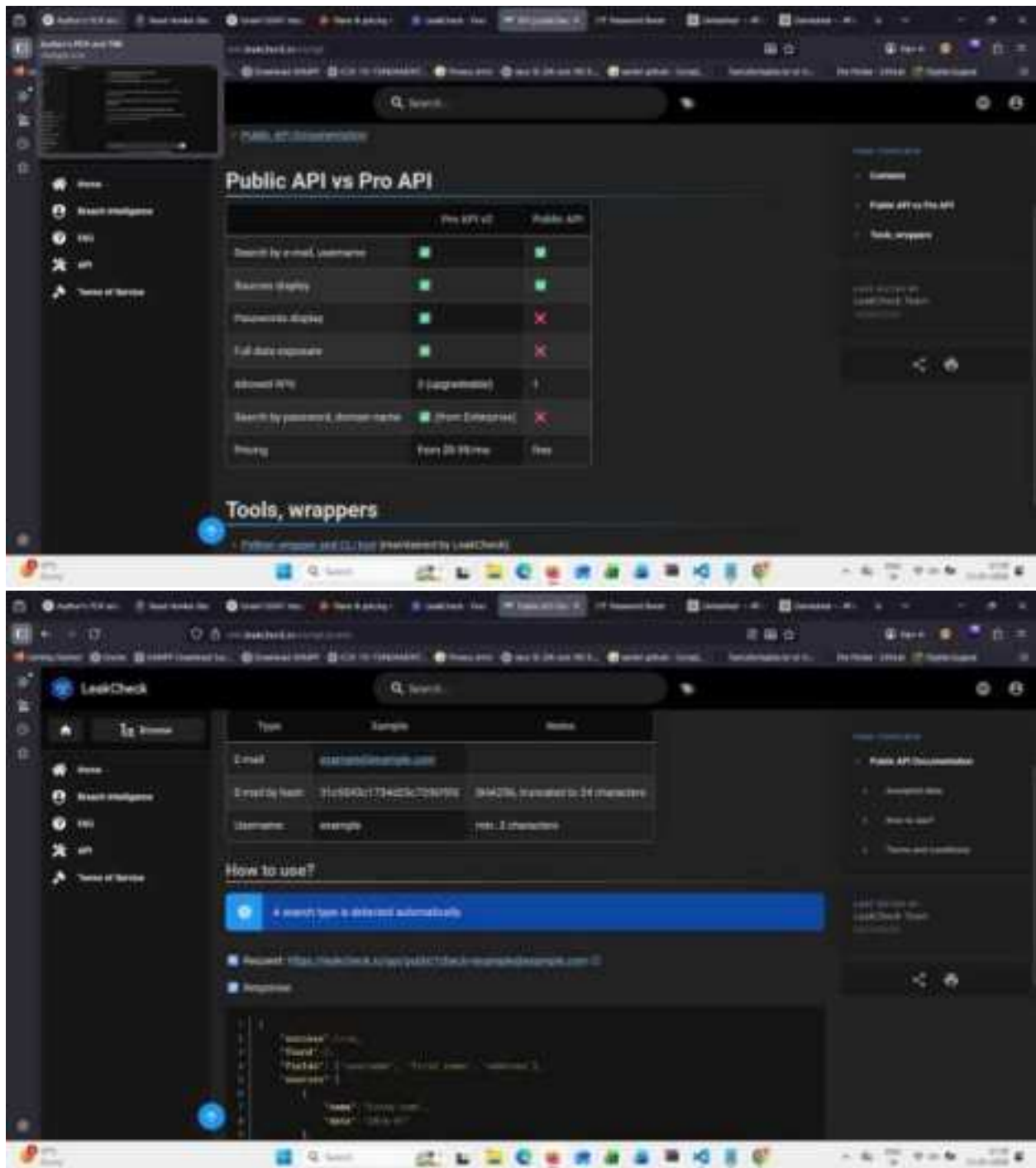
REFERENCE:

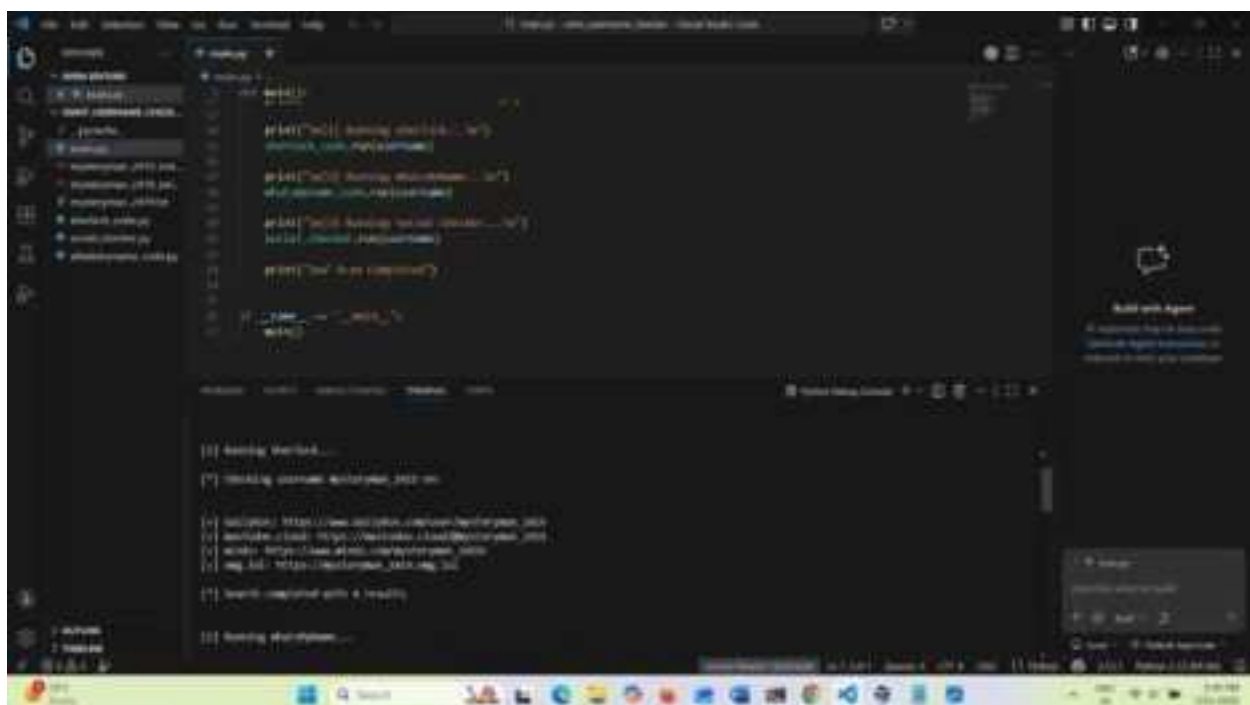
- <https://github.com/megadose/holehe>
- <https://whatsmyname.me/>
- <https://medium.com/@varppi/email-osint-techniques-c1e82efb253d>
- <https://hackers-arise.com/open-source-intelligenceosint-sherlock-the-ultimate-username-enumeration-tool/>
- <https://realpython.com>

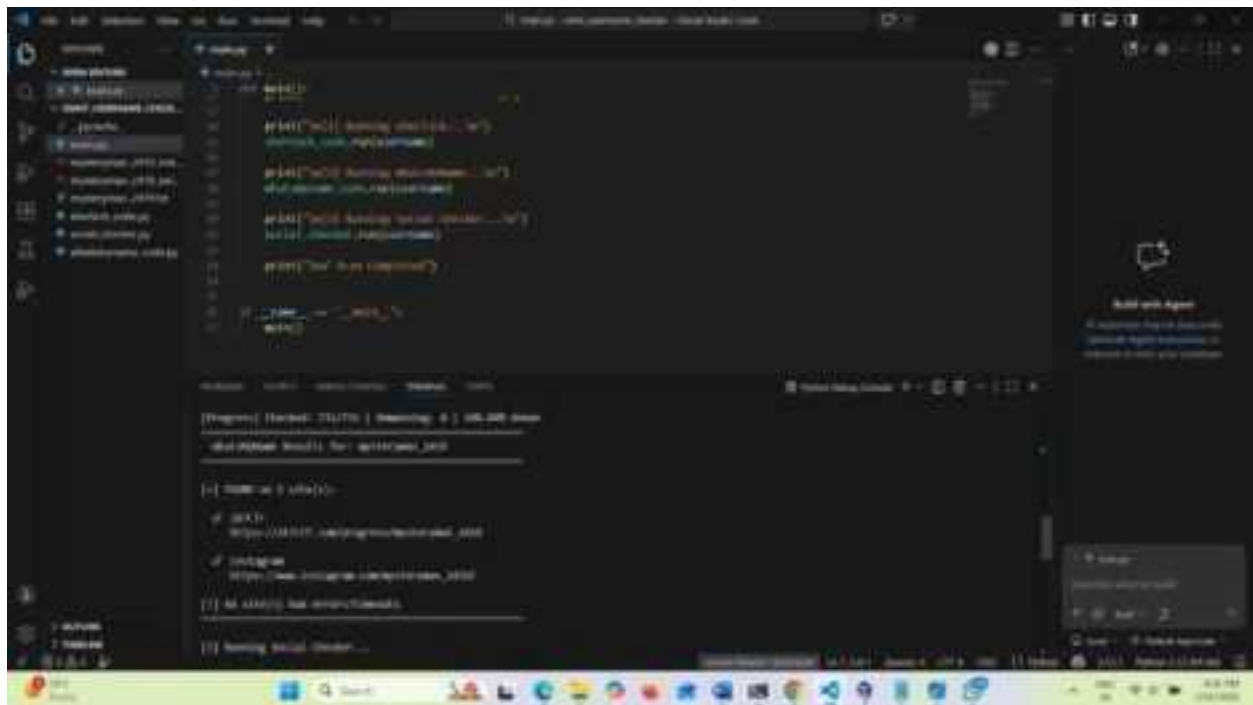
SCREEN SHOTS:

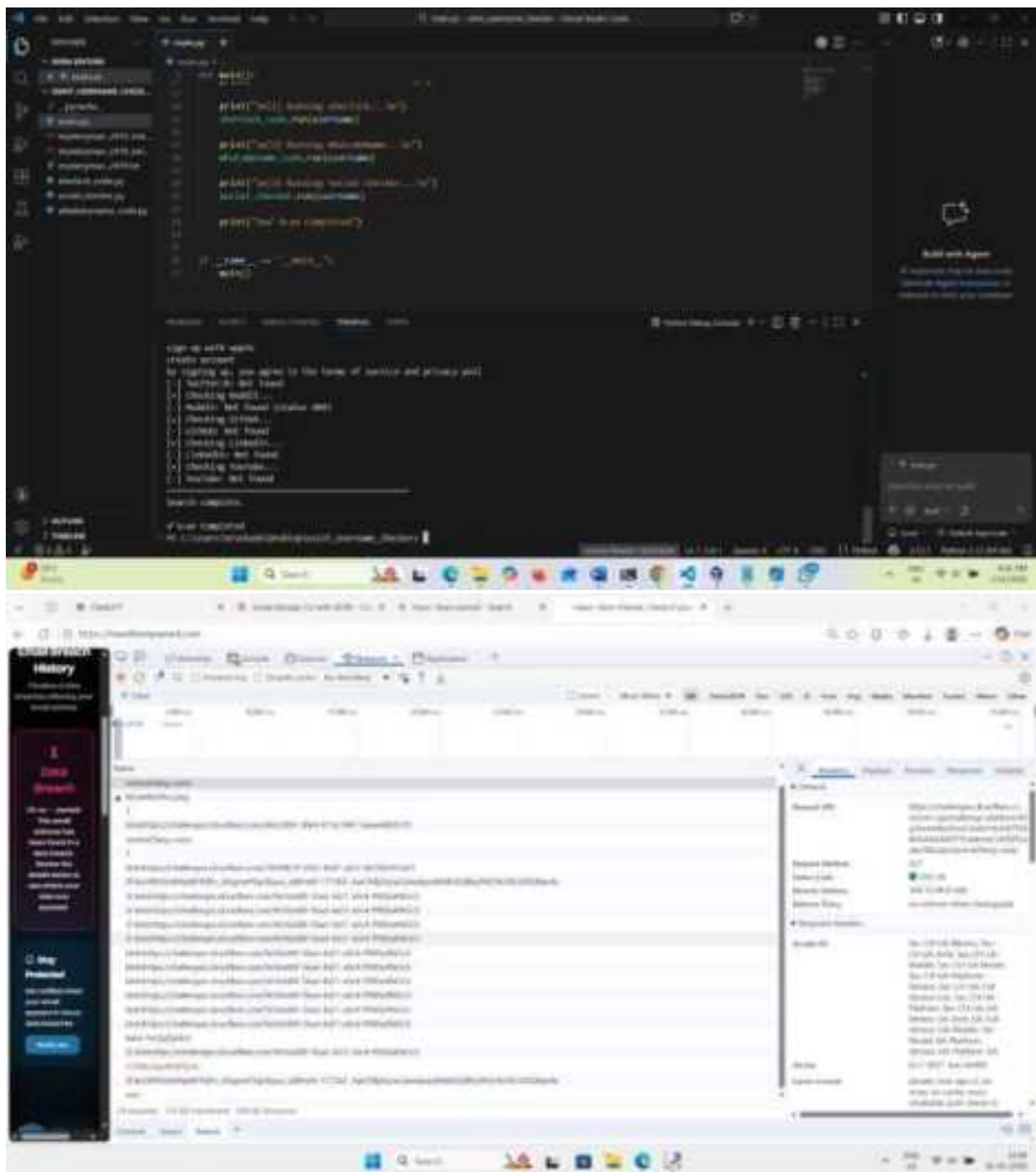












```
def __init__(self, model, data_loader, validation_loader, device):
    self.model = model
    self.data_loader = data_loader
    self.validation_loader = validation_loader
    self.device = device

def train(self):
    """Train the model"""
    # Create a DataLoader for the training data
    data_loader = DataLoader(self.data_loader.dataset, batch_size=10, shuffle=True)

    # Create a DataLoader for the validation data
    validation_loader = DataLoader(self.validation_loader.dataset, batch_size=10, shuffle=False)

    # Create a Loss function
    criterion = nn.LossFunction()

    # Create an Optimizer
    optimizer = optim.Adam(self.model.parameters())

    # Train the model
    for epoch in range(10):
        # Iterate over the training data
        for data, target in data_loader:
            # Move data and target to the device
            data, target = data.to(self.device), target.to(self.device)

            # Forward pass
            output = self.model(data)

            # Calculate loss
            loss = criterion(output, target)

            # Backward pass
            loss.backward()

            # Update parameters
            optimizer.step()

            # Zero the parameter gradients
            optimizer.zero_grad()

        # Iterate over the validation data
        validation_loss = 0
        for data, target in validation_loader:
            data, target = data.to(self.device), target.to(self.device)

            output = self.model(data)

            loss = criterion(output, target)

            validation_loss += loss.item()

        validation_loss /= len(validation_loader)

        # Print the training and validation loss
        print(f'Epoch {epoch+1}: Training Loss: {loss.item()}, Validation Loss: {validation_loss}')

    # Save the model
    torch.save(self.model.state_dict(), 'model.pth')

    # Load the model
    self.model.load_state_dict(torch.load('model.pth'))
```

```
def __init__(self, model, data_loader, validation_loader, device):
    self.model = model
    self.data_loader = data_loader
    self.validation_loader = validation_loader
    self.device = device

def train(self):
    """Train the model"""
    # Create a DataLoader for the training data
    data_loader = DataLoader(self.data_loader.dataset, batch_size=10, shuffle=True)

    # Create a DataLoader for the validation data
    validation_loader = DataLoader(self.validation_loader.dataset, batch_size=10, shuffle=False)

    # Create a Loss function
    criterion = nn.LossFunction()

    # Create an Optimizer
    optimizer = optim.Adam(self.model.parameters())

    # Train the model
    for epoch in range(10):
        # Iterate over the training data
        for data, target in data_loader:
            # Move data and target to the device
            data, target = data.to(self.device), target.to(self.device)

            # Forward pass
            output = self.model(data)

            # Calculate loss
            loss = criterion(output, target)

            # Backward pass
            loss.backward()

            # Update parameters
            optimizer.step()

            # Zero the parameter gradients
            optimizer.zero_grad()

        # Iterate over the validation data
        validation_loss = 0
        for data, target in validation_loader:
            data, target = data.to(self.device), target.to(self.device)

            output = self.model(data)

            loss = criterion(output, target)

            validation_loss += loss.item()

        validation_loss /= len(validation_loader)

        # Print the training and validation loss
        print(f'Epoch {epoch+1}: Training Loss: {loss.item()}, Validation Loss: {validation_loss}')

    # Save the model
    torch.save(self.model.state_dict(), 'model.pth')

    # Load the model
    self.model.load_state_dict(torch.load('model.pth'))
```

